Geavanceerde Computerarchitectuur
Session 4

Christoph Sanders & Wout Corijn

2025-03-03

1 Probleem

In dit labo worden twee aspecten van GPU-programmering onderzocht: geordende (coalesced) geheugentoegang en verschillende types GPU-geheugen. Dit onderzoek wordt gedaan op een Nvidea GTX 1050.

1.1 Deel 1: Geordende (coalesced) geheugen toegang

Het doel is om twee CUDA-kernels te ontwerpen die de rode kleurcomponent van een afbeelding inverteren met de formule **R=255-R**. Er worden twee versies geïmplementeerd: één met geordende - **(coalesced)** en één met ongeordende **(uncoalesced)** geheugentoegang. De uitvoeringstijd van beide versies wordt gemeten en vergeleken bij verschillende invoergroottes. Tot slot wordt de invloed van geheugenorganisatie op de prestaties geanalyseerd om conclusies te trekken over de efficiëntie van coalesced memory access.

1.2 Deel 2: Globaal, gedeeld en constant geheugen

Dit deel richt zich op het ontwerpen en analyseren van CUDA-kernels voor matrixvermenigvuldiging met verschillende types GPU-geheugen. Eerst wordt geheugen voor de matrices A, B en C gedefinieerd en gealloceerd. Vervolgens worden drie CUDA-kernels ontwikkeld: één die alleen **globaal geheugen** gebruikt, één die zowel **globaal en gedeeld geheugen** benut, en één die **globaal en constant geheugen** toepast. De uitvoeringstijden worden gemeten en de prestaties worden geanalyseerd voor verschillende invoergroottes.

2 Methode

2.1 Deel 1: Geordende (coalesced) geheugen toegang

2.1.1 Coalesced Geheugentoegang

Bij de coalesced implementatie wordt de rode kleurwaarde van de afbeelding geïnverteerd door threads lineair toegang te geven tot de globale geheugenlocaties. De threads accessen opeenvolgende geheugenadressen, wat zorgt voor coalesced geheugentoegang.

Bij onderstaande implementatie wordt de **image_array** voorgaand eerst gesorteerd als **RR...RGG..GBB...B**.

```
__global__ void convertRed_coalesced(uint8_t* image_array, int size) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    if(index < size / 3) {
        image_array[index] = 255 - image_array[index];
    }
}
```

2.1.2 Un-Coalesced Geheugentoegang

Bij un-coalesced geheugen zijn de geheugenlocaties niet aaneengesloten, wat de efficiëntie van de kernel vermindert. Elke thread verwerkt een specifieke kleurcomponent, waardoor de geheugentoegang inefficiënter is en de prestaties afnemen.

```
__global__ void convertRed_uncoalesced(uint8_t* image_array, int size) {
    int index = (blockIdx.x * blockDim.x + threadIdx.x) * 3;
    if(index < size) {
        image_array[index] = 255 - image_array[index];
    }
}
```

3 Deel 2: Globaal, gedeeld en constant geheugen

Het vermenigvuldigen van matrices kan op verschillende manieren worden uitgevoerd. Hierbij kan gebruik worden gemaakt van enkel globaal of globaal samen met gedeeld en/of constant geheugen.

3.1 Enkel-globaal geheugen

Voordelen:

- De grootte van dit geheugen bedraagt enkele gigabytes, waardoor de hele matrix hierin kan worden opgeslagen.
- Globaal geheugen kan worden gelezen en beschreven door alle threads, zelfs door threads in verschillende blokken.

Nadelen:

- Als de geheugentoegangen niet coalesced zijn, zullen de lees- en schrijfbewerkingen langer duren.

3.2 Globaal en gedeeld geheugen

Gedeeld geheugen (shared memory) is geheugen dat toegankelijk is voor threads binnen eenzelfde blok.

Voordelen:

- Veel sneller dan globaal geheugen.

Nadelen:

- Een zeer beperkte grootte van slechts 48 KB per multiprocessor [1].
- Alleen threads binnen eenzelfde blok kunnen toegang krijgen tot het gedeelde geheugen.

3.3 Globaal en constant geheugen

Constant geheugen is geheugen dat bedoeld is voor read-only data en wordt gedeeld tussen alle threads.

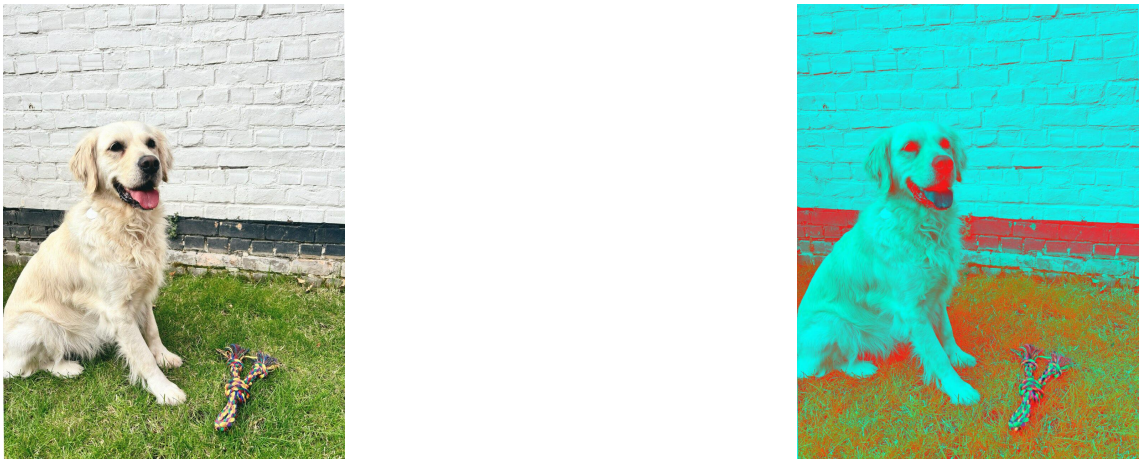
Voordelen:

- Dit geheugen kan snel worden gelezen.

Nadelen:

- Een beperkte grootte van slechts 64 KB. [2]
- Kan alleen worden gelezen, alleen de CPU kan naar dit geheugen schrijven.

4 Resultaat



(a) Normaal

(b) Inversie rood

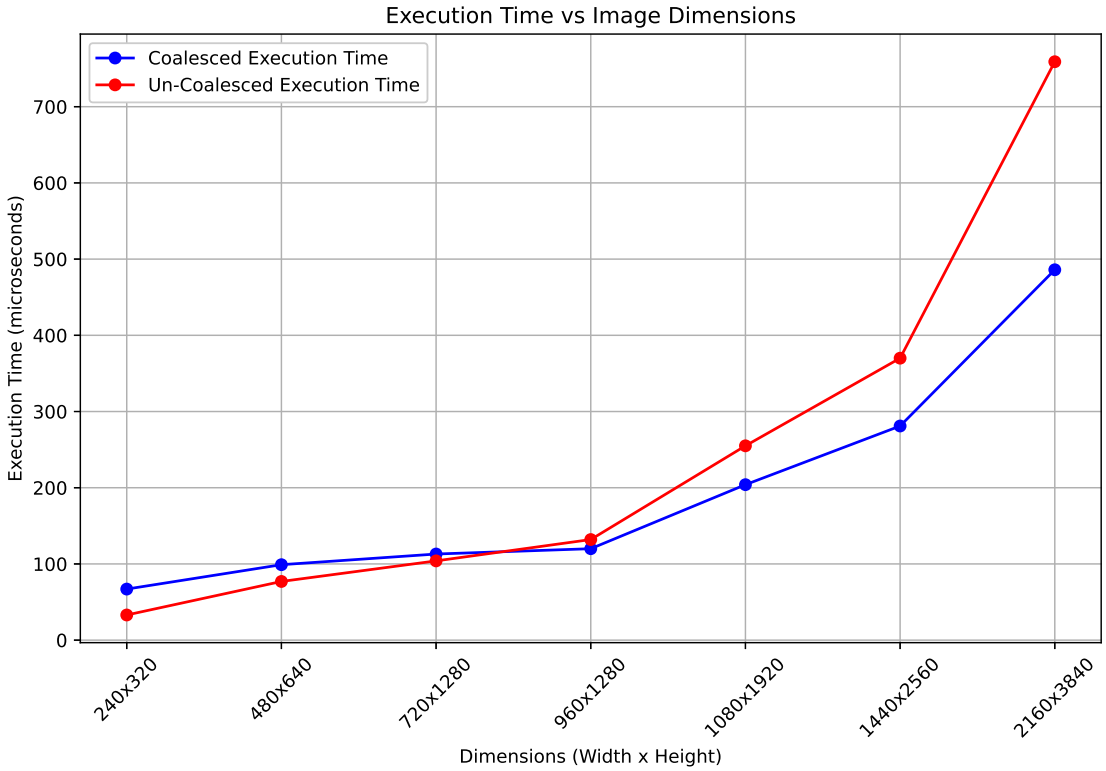
Figuur 1: afbeeldingen

4.1 Deel 1: Geordende (coalesced) geheugen toegang

Figuur 2 toont de gemiddelde uitvoeringstijd van 20 runs voor het inverteren van de rode kleur van een afbeelding bij verschillende beeldresoluties.

Un-Coalesced Memory Access (rode lijn): Bij lage beeldresoluties presteert deze methode beter dan coalesced memory access. Dit komt door de overhead van kernel-initialisatie en geheugentoegang, die een relatief groot effect heeft bij kleinere datasets. Naarmate de resolutie toeneemt, neemt de efficiëntie van un-coalesced toegang af als gevolg van onsamenhangende geheugentoegangen, wat leidt tot een stijging in de uitvoeringstijd.

Coalesced Memory Access (blauwe lijn): Deze methode heeft bij kleinere afbeeldingsresoluties initieel een hogere uitvoeringstijd vertoont als gevolg van de overhead die gepaard gaat met geheugenoptimalisatie, wordt het prestatievoordeel duidelijker bij toenemende resolutie. Doordat threads binnen een warp opeenvolgende geheugenadressen aanspreken, wordt het geheugen efficiënter benut, wat resulteert in lagere uitvoeringstijden bij grotere afbeeldingen.



Figuur 2: uitvoeringstijd vs afbeeldingsgrootte

5 Deel 2: Globaal, gedeeld en constant geheugen

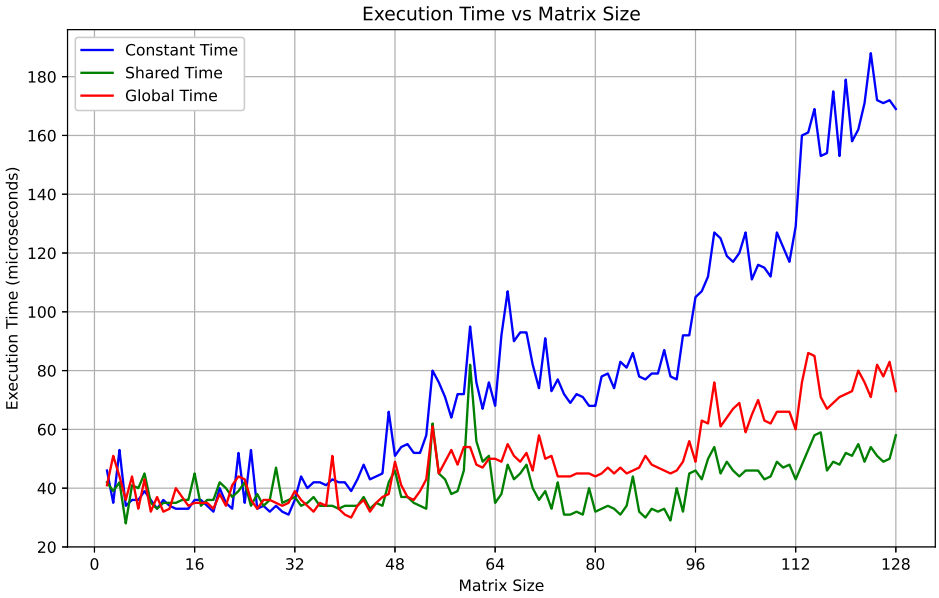
De grafiek in Figuur 3 toont de uitvoeringstijd (in microseconden) als functie van de matrixgrootte voor constant (blauw), gedeeld (groen) en globaal geheugen (rood). Voor elke meting zijn **1000 runs per kernel** uitgevoerd, met matrixgroottes van 2x2 tot 128x128.

5.1 Hoe vergelijken de uitvoeringstijden van matrixvermenigvuldiging zich bij het gebruik van de drie verschillende geheugentypes?

- **Enkel globaal geheugen (rood)**: De uitvoeringstijd van globaal geheugen blijft in de grafiek relatief stabiel in vergelijking met constant geheugen (blauw), vooral bij grotere matrixgroottes. Hoewel het iets hoger ligt dan gedeeld geheugen (groen), toont het een minder sterke toename bij toenemende matrixgrootte. Geheugencoalescing helpt om de prestaties te verbeteren, doordat opeenvolgende threads toegang hebben tot aaneengesloten geheugenlocaties, wat schommelingen beperkt.

- **Globaal en gedeeld geheugen (groen)**: Gedeeld geheugen wordt gedeeld binnen een CUDA-block. Het heeft een lage latency en maakt gebruik van datasharing tussen threads, waardoor data efficiënt hergebruikt wordt en minder geheugenvertraging ontstaat.

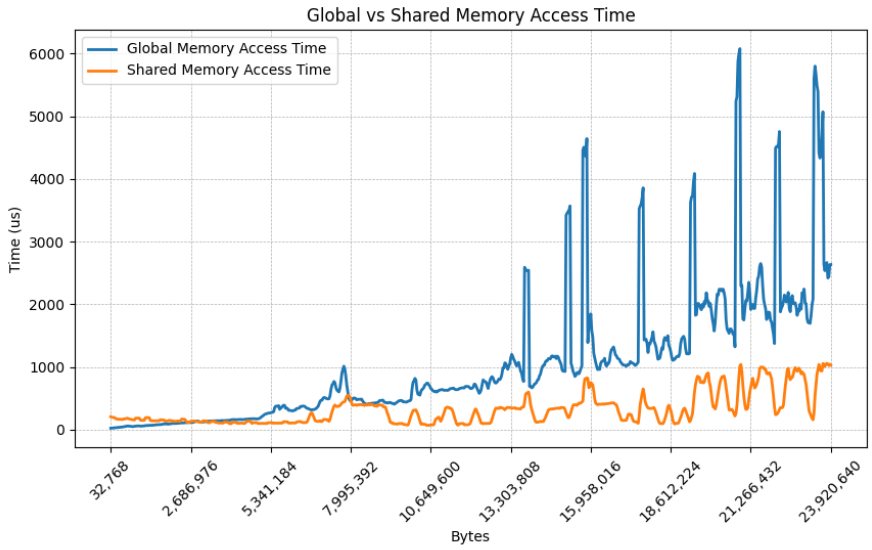
- **Globaal en constant geheugen (blauw)**: Dit geheugen is geoptimaliseerd voor situaties waarin alle threads dezelfde waarde lezen. In deze metingen leest echter elke thread verschillende elementen, wat leidt tot *cache misses* en extra globale geheugenfetches. Hierdoor schaalt constant geheugen slecht voor grote matrices. Het is vooral geschikt voor kleine, vaak gebruikte gedeelde waarden, niet voor grote matrices waarbij threads verschillende delen aanspreken.



Figuur 3: uitvoeringstijd vs matrixgrootte

5.2 Hoeveel tijd kost het om gegevens over te zetten (van CPU naar GPU globaal geheugen; van GPU globaal geheugen naar GPU gedeeld geheugen)?

Het verplaatsen van gegevens van globaal naar gedeeld geheugen zal bij grote datasets sneller verlopen, aangezien het gedeelde geheugen parallel wordt gevuld. Zolang de blokken parallel verwerkt kunnen worden, wordt geen prestatieverlies verwacht. De overdracht van de CPU naar het globale geheugen van de GPU verloopt echter sequentieel, en de tijdsduur hiervan zal toenemen naarmate de dataset groter wordt. Dit kan ook worden afgeleid uit Figuur 4.



Figuur 4: CPU memory to Global vs Global to Shared

Bibliografie

[1] R. Crovella, “Find the limit of shared memory that can be used per block,” 2017. Geraadpleegd op: 6 maart 2025.

[2] njuffa, “Sm,share memory size,constant size?,” 2017. Geraadpleegd op: 6 maart 2025.

6 Appendix

6.1 Deel 1: Geordende (coalesced) geheugen toegang

6.1.1 Invert red - Coalesced

```
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <chrono>

#define M 960       // VR width
#define N 1280       // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

uint8_t* get_image_array(void){
...
}


void save_image_array(uint8_t* image_array){
...
}

__global__ void reformat(uint8_t* image_array,uint8_t* out_image_array, int size) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
        i < size;
        i += blockDim.x * gridDim.x)
    {
        if(i%3 == 0){
            out_image_array[i/3] = image_array[i];
        }
        else if(i%3 == 1){
            out_image_array[(i-1)/3 + size/3] = image_array[i];
        }
        else if(i%3 == 2){
            out_image_array[(i-2)/3 + size*2/3] = image_array[i];
        }
    }
}

__global__ void deformat(uint8_t* image_array,uint8_t* out_image_array, int size) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
        i < size;
        i += blockDim.x * gridDim.x)
    {
        if(i < size/3){
            out_image_array[i*3] = image_array[i];
        }
        else if(i < size*2/3){
            int index = i - size/3;
            out_image_array[index*3 + 1] = image_array[i];
        }
        else if(i < size){
            int index = i - size*2/3;
            out_image_array[index*3 + 2] = image_array[i];
        }
    }
}


```

```
__global__ void convertRed_coalesced(uint8_t* image_array, int size) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    image_array[index] = 255 - image_array[index];
}

int main (void) {
    // Read the image
    uint8_t* image_array = get_image_array();


    uint8_t *d_image_array;
    uint8_t *d_temp_image_array;
    int size = M*N*C;

    // red pixels invert
    int threads = size/3;

    cudaMalloc((void**)&d_image_array, size * sizeof(uint8_t));
    cudaMalloc((void**)&d_temp_image_array, size * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_temp_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    int threadsPerBlock = 1024;
    int blocks = (threads + threadsPerBlock - 1) / threadsPerBlock;

    reformat<<<blocks, threadsPerBlock>>>(d_image_array, d_temp_image_array, size);
    // Synchronize cuda
    cudaDeviceSynchronize();
    printf("threads: %d, blocks: %d\n", threadsPerBlock, blocks);
    auto start = std::chrono::high_resolution_clock::now();
    convertRed_coalesced<<<blocks, threadsPerBlock>>>(d_temp_image_array, size);
    cudaDeviceSynchronize();
    auto end = std::chrono::high_resolution_clock::now();

    deformat<<<blocks, threadsPerBlock>>>(d_temp_image_array, d_image_array, size);

    cudaMemcpy(image_array, d_image_array, size* sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    cudaFree(d_image_array);
    cudaFree(d_temp_image_array);


    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end -
        start);
    std::cout << "Time difference: " << duration.count() << " microseconds" << std
        ::endl;

    // Save the image
    save_image_array(image_array);

    return 0;
}
```

6.1.2 Invert red - Un-coalesced

```

#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <chrono>

#define M 960       // VR width
#define N 1280       // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length

uint8_t* get_image_array(void){
...
```

```
}


void save_image_array(uint8_t* image_array){
...
}


__global__ void convertRed_uncoalesced(uint8_t* image_array, int size) {
    int index = (blockIdx.x * blockDim.x + threadIdx.x) *3;
    image_array[index] = 255 - image_array[index];
}

int main (void) {
    // Read the image
    uint8_t* image_array = get_image_array();


    uint8_t *d_image_array;
    uint8_t *d_temp_image_array;
    int size = M*N*C;

    // red pixels invert
    int threads = size/3;

    cudaMalloc((void**)&d_image_array, size * sizeof(uint8_t));
    cudaMalloc((void**)&d_temp_image_array, size * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_temp_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    int threadsPerBlock = 1024;
    int blocks = (threads + threadsPerBlock - 1) / threadsPerBlock;

    // Synchronize cuda
    cudaDeviceSynchronize();
    printf("threads: %d, blocks: %d\n", threadsPerBlock, blocks);
    auto start = std::chrono::high_resolution_clock::now();
    convertRed_uncoalesced<<<blocks, threadsPerBlock>>>(d_image_array, size);
    cudaDeviceSynchronize();
    auto end = std::chrono::high_resolution_clock::now();

    cudaMemcpy(image_array, d_image_array, size* sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    cudaFree(d_image_array);


    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end -
        start);
    std::cout << "Time difference: " << duration.count() << " microseconds" << std
        ::endl;

    // Save the image
    save_image_array(image_array);

    return 0;
}
```

6.2 Deel 2: Globaal, gedeeld en constant geheugen

```
#include <cstdint>
#include <iostream>
#include <chrono>
#include <fstream>
#include <cuda_runtime.h>

#define MAX_SIZE 128
```

```
#define BLOCK_SIZE 32
#define RUNS 100

__constant__ uint8_t d_matrix1[MAX_SIZE * MAX_SIZE];
__constant__ uint8_t d_matrix2[MAX_SIZE * MAX_SIZE];

__global__ void mulMatrixConstant(uint8_t* d_matrix3, int size) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int indexX = index % size;
    int indexY = index / size;

    if (indexX < size && indexY < size) {
        int sum = 0;
        for (int i = 0; i < size; i++) {
            sum += d_matrix1[indexY * size + i] * d_matrix2[i * size + indexX];
        }
        d_matrix3[indexY * size + indexX] = sum;
    }
}

__global__ void mulMatrixShared(uint8_t* d_matrix1, uint8_t* d_matrix2, uint8_t*
    d_matrix3, int size) {
    __shared__ uint8_t tileA[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ uint8_t tileB[BLOCK_SIZE][BLOCK_SIZE];

    int indexX = threadIdx.x + blockIdx.x * blockDim.x;
    int indexY = threadIdx.y + blockIdx.y * blockDim.y;

    if (indexX >= size || indexY >= size) return;

    int sum = 0;
    for (int k = 0; k < (size + BLOCK_SIZE - 1) / BLOCK_SIZE; k++) {
        if (indexY < size && (k * BLOCK_SIZE + threadIdx.x) < size)
            tileA[threadIdx.y][threadIdx.x] = d_matrix1[indexY * size + k *
                BLOCK_SIZE + threadIdx.x];
        else
            tileA[threadIdx.y][threadIdx.x] = 0;

        if (indexX < size && (k * BLOCK_SIZE + threadIdx.y) < size)
            tileB[threadIdx.y][threadIdx.x] = d_matrix2[(k * BLOCK_SIZE + threadIdx
                .y) * size + indexX];
        else
            tileB[threadIdx.y][threadIdx.x] = 0;

        __syncthreads();

        for (int i = 0; i < BLOCK_SIZE; i++) {
            sum += tileA[threadIdx.y][i] * tileB[i][threadIdx.x];
        }
    }
    d_matrix3[indexY * size + indexX] = sum;
}

__global__ void mulMatrixGlobal(uint8_t* d_matrix1, uint8_t* d_matrix2, uint8_t*
    d_matrix3, int size) {
    int indexX = threadIdx.x + blockIdx.x * blockDim.x;
    int indexY = threadIdx.y + blockIdx.y * blockDim.y;

    if (indexX < size && indexY < size) {
        int sum = 0;
        for (int i = 0; i < size; i++) {
            sum += d_matrix1[indexY * size + i] * d_matrix2[i * size + indexX];
        }
        d_matrix3[indexY * size + indexX] = sum;
    }
}

void testMatrixSize(int size, std::ofstream& csvFile, long long& timeConstant, long
     long& timeShared, long long& timeGlobal) {
    uint8_t *h_matrix1 = new uint8_t[size * size];
```

```
    uint8_t *h_matrix2 = new uint8_t[size * size];
    uint8_t *h_matrix3 = new uint8_t[size * size]();
    uint8_t *d_matrix1, *d_matrix2, *d_matrix3;

    for (int i = 0; i < size * size; i++) {
        h_matrix1[i] = 1;
        h_matrix2[i] = 1;
    }

    cudaMalloc((void**)&d_matrix3, size * size * sizeof(uint8_t));

    cudaMalloc((void**)&d_matrix1, size * size * sizeof(uint8_t));
    cudaMalloc((void**)&d_matrix2, size * size * sizeof(uint8_t));
    cudaMemcpy(d_matrix1, h_matrix1, size * size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_matrix2, h_matrix2, size * size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(BLOCK_SIZE, BLOCK_SIZE);
    dim3 blocksPerGrid((size + BLOCK_SIZE - 1) / BLOCK_SIZE, (size + BLOCK_SIZE -
        1) / BLOCK_SIZE);

    long long totalTime = 0;

    // Constant method
    cudaMemcpyToSymbol(d_matrix1, h_matrix1, size * size * sizeof(uint8_t));
    cudaMemcpyToSymbol(d_matrix2, h_matrix2, size * size * sizeof(uint8_t));
    totalTime = 0;
    for (int i = 0; i < RUNS; i++) {
        cudaDeviceSynchronize();
        auto start = std::chrono::high_resolution_clock::now();
        mulMatrixConstant<<<blocksPerGrid, threadsPerBlock>>>(d_matrix3, size);
        cudaDeviceSynchronize();
        auto end = std::chrono::high_resolution_clock::now();
        totalTime += std::chrono::duration_cast<std::chrono::microseconds>(end -
            start).count();
    }
    timeConstant = totalTime / RUNS;

    // Shared method
    totalTime = 0;
    for (int i = 0; i < RUNS; i++) {
        cudaDeviceSynchronize();
        auto start = std::chrono::high_resolution_clock::now();
        mulMatrixShared<<<blocksPerGrid, threadsPerBlock>>>(d_matrix1, d_matrix2,
            d_matrix3, size);
        cudaDeviceSynchronize();
        auto end = std::chrono::high_resolution_clock::now();
        totalTime += std::chrono::duration_cast<std::chrono::microseconds>(end -
            start).count();
    }
    timeShared = totalTime / RUNS;

    // Global method
    totalTime = 0;
    for (int i = 0; i < RUNS; i++) {
        cudaDeviceSynchronize();
        auto start = std::chrono::high_resolution_clock::now();
        mulMatrixGlobal<<<blocksPerGrid, threadsPerBlock>>>(d_matrix1, d_matrix2,
            d_matrix3, size);
        cudaDeviceSynchronize();
        auto end = std::chrono::high_resolution_clock::now();
        totalTime += std::chrono::duration_cast<std::chrono::microseconds>(end -
            start).count();
    }
    timeGlobal = totalTime / RUNS;

    // Write results to CSV
    std::cout << "Size:␣" << size << "x" << size << "␣-␣Constant:␣" << timeConstant
         << "␣Shared:␣" << timeShared << "␣Global:␣" << timeGlobal << std::endl;
```

```
134     csvFile << size << "," << timeConstant << "," << timeShared << "," <<
            timeGlobal << "\n";
135
136     cudaMemcpy(h_matrix3, d_matrix3, size * size * sizeof(uint8_t),
            cudaMemcpyDeviceToHost);
137     cudaFree(d_matrix3);
138     cudaFree(d_matrix1);
139     cudaFree(d_matrix2);
140
141     delete[] h_matrix1;
142     delete[] h_matrix2;
143     delete[] h_matrix3;
144 }
145
146 int main() {
147     std::ofstream csvFile("matrix_multiplication_times.csv");
148     csvFile << "Size,Constant␣(microseconds),Shared␣(microseconds),Global␣(
            microseconds)\n";
149
150     for (int size = 2; size <= MAX_SIZE; size += 1) {
151         long long timeConstant, timeShared, timeGlobal;
152         testMatrixSize(size, csvFile, timeConstant, timeShared, timeGlobal);
153     }
154
155     csvFile.close();
156     return 0;
157 }
```